

The New Low-Latency Block Cipher uLBC

Anonymous Submission

Abstract. In recent years, there has been a growing interest in low-latency ciphers. Since the first low-latency block cipher PRINCE was proposed at ASIACRYPT 2012, many low-latency primitives sprung up, such as Midori, MANTIS, QARMA and SPEEDY. Some ciphers, like SPEEDY and Orthros, introduce bit permutations to achieve reduced delay. However, this approach poses a challenge in evaluating the resistance against some cryptanalysis, especially differential and linear attacks. SPEEDY-7-192, was fully broken by Boura *et.al.* using differential attack, for example. In this paper, we manage to propose a novel low-latency block cipher, which guarantees security against differential and linear attacks. Revisiting the permutation technique used in Orthros, we investigate the selection of nibble permutations and propose a method for selecting them systematically rather than relying on random search. Our new nibble permutation method ensures the existence of impossible differential and differential trails for up to 8 rounds, while the nibble permutations for both branches of Orthros may lead to a 9-round impossible differential trail. Furthermore, we introduce a new approach for constructing low-latency coordinate functions for 4-bit S-boxes, which involves a more precise delay computation compared to traditional methods based solely on circuit depth. The new low-latency primitive uLBC we propose, is a family of 128-bit block ciphers, with three different versions of key length, respectively 128-bit and 256-bit key, as well as a 384-bit tweakey version with variable-length key. According to the key length, named uLBC-128, uLBC-256 and uLBC-384t. Our analysis shows that uLBC-128 exhibits lower latency and area requirements compared to ciphers such as QARMA₉-128 and Midori128. On performance, uLBC-128 has excellent AT performance, the best performance except SPEEDY-6, and even the best performance in UMC55nm in our experiments.

Keywords: Block Cipher Design · Low Latency Cipher · Low Latency S-box

1 Introduction

Block cipher is a fundamental cryptographic primitive, which is widely used to provide confidentiality in software and hardware systems. Designing a block cipher is a complex task that requires a carefully balanced strategy considering factors like cryptographic strength, latency, implementation cost, power, and energy consumption. The choice of trade-offs depends on the specific application requirements.

Low-latency ciphers have gained significant attention in recent years and offer notable advantages in high-throughput scenarios, including the Internet of Things (IoT) and encrypted hardware components of CPUs. In 2012, Borghoff *et al.* introduced the first low-latency block cipher named PRINCE [BCG⁺12], which proposes the α -reflection structure where the encryption and decryption are similar with the keys K and $K \oplus \alpha$, respectively. The round function of PRINCE resembles the structure of AES, except that it incorporates 4×4 almost MDS binary matrices with a branch number of 4, instead of the MDS matrix in the MixColumns mapping. Subsequently, a series of ciphers were introduced, such as Midori [BBI⁺15], MANTIS [BJK⁺16], QARMA [Ava17] and SPEEDY [LMMR21] etc.

Midori [BBI⁺15], introduced at ASIACRYPT 2015, is a family of two block ciphers: Midori64 and Midori128, each with a different block length. Midori128 applies an AES-like design, with the 8-bit S-boxes as the confusion module and 4×4 almost MDS binary

matrices as the diffusion module. The 8-bit S-boxes consist of two 4-bit S-boxes processed in parallel with bit permutation both in input and output, which can minimize the path delay in the round-based implementation. There are some cryptanalysis results that threaten the security of Midori. Guo *et al.* gave the invariant subspace attack against the full block cipher Midori64 [GJN⁺15]. G rault *et al.* gave a practical attack on Midori64 and Midori128 under related key model [GL16]. MANTIS with 64-bit block length was introduced in CRYPTO 2016 [BJK⁺16], as a low-latency variant of the SKINNY family. MANTIS is a tweakable block cipher used for memory encryption, incorporating a TWEAKEY schedule [JNP14] derived from PRINCE. It also employs the involutory of Midori’s S-box to optimize for small areas and low circuit depth. QARMA [Ava17] draws inspiration from PRINCE and MANTIS but employs a three-round Even-Mansour scheme rather than an FX-construction, which was improved to QARMA V2 with longer tweaks and improved security bounds [ABD⁺23]. In SAC 2020, the authors of PRINCE introduced PRINCE v2 [BEK⁺21], which improves the security of PRINCE without altering the number of rounds or rotation operations. Recently, some new low-latency block ciphers have been proposed. SPEEDY [LMMR21] incorporates CMOS hardware into the cipher design and applies a high-speed 6-bit S-box implemented using NAND gates. SPEEDY has a low delay for encryption without any requirements for area and decryption speed. However, Boura *et al.* give a differential attack for the full round SPEEDY-7-192, which breaks the expected security strength [BDHBNP23]. For some special application, Belkheyar *et al.* [BDD⁺23] designed a 24-bit low-latency tweakable block cipher with a 40-bit tweak, specifically for a memory safety concept called Cryptographic Capability Computing. In addition to the block ciphers, Banik *et al.* introduced a low-latency pseudorandom function named Orthros [BIL⁺21], which takes a 128-bit message and a 128-bit key as input and produces a 128-bit random number as output. Orthros is constructed using two 128-bit low-latency permutations. To minimize latency, the permutation involves 4-bit S-boxes, bit permutation, nibble permutation, and 4×4 almost MDS (Maximum Distance Separable) binary matrices.

In the context of the low-latency block cipher, it is necessary to construct S-boxes that have low-latency. There are many methods for the selection of low-latency S-boxes [BGLS19, Ras22]. When constructing low-latency S-boxes, designers can use different metrics to measure the latency of coordinate functions. The most popular metric is depth, which assesses the path delay of the S-box by considering the sequential performance of the involved transistors. However, it treats NAND and NOR equally, which needs to be improved, just as analyzed in [LMMR21].

Our contribution. This paper proposes a new 128-bit block cipher while providing a strong security argument for its resistance against differential and linear attacks, which has the lowest latency among the 128-bit block cipher as far as we know.

Among the low-latency ciphers, SPEEDY applies low-latency 6-bit S-box, with a complex diffusion function, which makes it hard to give an accurate estimation of the probability of differential paths and linear paths. Midori128 and QARMA-128 both employ a byte-based design similar to AES, which facilitates establishing the lower bound for the active S-boxes. The round function utilizes a low-latency 8-bit S-box composed of two 4-bit S-boxes operating in parallel. The diffusion layer utilizes a selective byte permutation and an almost 4×4 MDS matrix, with a branch number of 4. For the low-latency pseudorandom function (PRF) Orthros, which adopts a 128-bit permutation, a 32-nibble permutation, and the above almost MDS binary matrix. The use of bit permutation in the cipher makes it a challenge to find the upper bound of the optimal differential characteristic. Therefore, we specifically focus on the 32-nibble permutation for the security proof.

We use the 4-bit S-box, nibble permutation, and the almost 4×4 MDS matrix to construct the block cipher. This transformation has a branch number of 4, allowing an

active nibble to propagate to 27 nibbles at most after three rounds. Therefore, the aim is to have an active nibble that can diffuse to any of the 32 nibbles (a case commonly referred to as full diffusion). It is desired to have a round number of at least 4. In this case, the selection of a nibble permutation resistant to the differential attack is of great importance. We propose a well-chosen method for the nibble permutation instead of the random searching of the *Orthros* which can guarantee both the impossible differential and differential characteristic to be present 8 rounds at most.

4-bit S-boxes are employed to optimize the circuit's depth. A new method for constructing low-latency coordinate functions is proposed. Utilizing the appropriate combination of coordinate functions, a large number of low-latency S-boxes are generated. Then we perform cryptographic analysis on these S-boxes and select one with the optimal security characteristics. We hope that our findings will inspire others to design new and efficient low-latency cryptographic primitives.

Besides, we give an optimal lightweight key schedule, which supports the extension of tweak block cipher following the TWEAKEY framework [JNP14]. We give a low-latency version of *uLBC-128* and *uLBC-256*, which are given proof of resistance against differential and linear attacks. A secure tweakable cipher version with a 384-bit length of tweak and keys against related-key attacks is presented, which supports variable key lengths from 128-384 bits.

We give the full unrolled hardware implementation for ciphers *Midori*, *QARMA*, *AES*, etc. Among them, *uLBC-128* is the lowest latency among the 128-bit block ciphers with the same security claim. On performance, *uLBC-128* has excellent AT performance, the best performance except *SPEEDY-6*, and even the best performance in UMC 55nm in our experiments. Our implementations are publicly available in <https://anonymous.4open.science/r/Low-Latency-D6CF>.

2 Descriptions of Block Cipher *uLBC*

The *uLBC* is a low-latency block cipher based on Substitution Permutation Networks (SPNs) with proof of security against differential and linear attacks. In this section, we first give an overview of *uLBC*, followed by an introduction to encryption and decryption algorithms and the key schedule algorithms.

2.1 Overview

The block cipher *uLBC* is a 128-bit block with 128-bit or 256-bit key lengths. Depending on the key lengths, we refer to them as *uLBC-128* and *uLBC-256*. For the security of block ciphers, especially in low-latency applications, we primarily consider the security under a single-key setting. The security under the single-key setting is referred to as SK-security. The low latency ciphers *Midori*, *QARMA*, *PRINCE*, and *SPEEDY* all maintain SK security. Hence, we can reduce the rounds of the *uLBC* to obtain high-performance versions for SK-security. Meanwhile, we provide a version *uLBC-384t* which resists related-tweak attack and even stronger, related-key attack. This property is referred to as RK-security. The relevant parameters of each version are shown in Table 1.

Table 1: The parameters of *uLBC*.

Scheme	Block size	Key size	Round	Security
<i>uLBC-128</i>	128 bits	128 bits	18	SK-security
<i>uLBC-256</i>	128 bits	256 bits	22	SK-security
<i>uLBC-384t</i>	128 bits	128-384 bits	30	RK-security

The *uLBC* computation follows a big-endian format. The 128-bit block $b_0, \dots, b_{127}$

is divided into 32 nibbles, $s_0 = b_0b_1b_2b_3$, $s_1 = b_4b_5b_6b_7$, ..., $s_{31} = b_{124}b_{125}b_{126}b_{127}$. The nibble is the basic unit referred to as a cell. The block includes 32 cells organized in a 4 by 8 matrix, as illustrated in the following.

$$\begin{pmatrix} s_0 & s_4 & s_8 & s_{12} & s_{16} & s_{20} & s_{24} & s_{28} \\ s_1 & s_5 & s_9 & s_{13} & s_{17} & s_{21} & s_{25} & s_{29} \\ s_2 & s_6 & s_{10} & s_{14} & s_{18} & s_{22} & s_{26} & s_{30} \\ s_3 & s_7 & s_{11} & s_{15} & s_{19} & s_{23} & s_{27} & s_{31} \end{pmatrix}.$$

2.2 Encryption

The block cipher uLBC is based on the SPN structure. First, the plaintext state P is exclusive-ored with the whitening key RK_0 to get an intermediate state, followed by N rounds of round functions. The **round function** includes S-box substitution (**SubNib**), constant addition (**AddConst**), cell permutation (**PosPerm**), column mixing (**MixColumn**), and round key addition (**AddRK**) operations. The round function is illustrated in Figure 1 and the encryption is given in Algorithm 1. The $RK_1, RK_2, \dots, RK_N$ are round keys generated by the key schedule algorithm. The final output is the ciphertext C . We provide test vectors of uLBC in Appendix A.

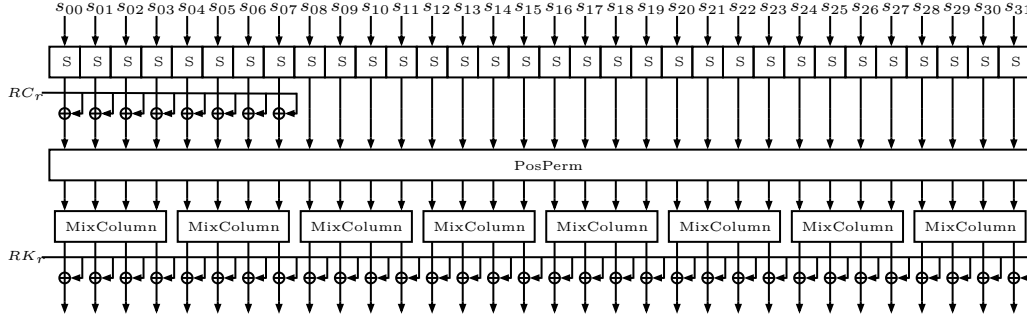


Figure 1: The round function of uLBC.

Algorithm 1 The Encryption of uLBC.

Input: plaintext: P , round key: $RK_0, \dots, RK_N$

Output: ciphertext: C

```

 $X_0 = P \oplus RK_0$  ▷ XORed with the whitening key
for  $i = 0$  to  $N - 2$  do ▷ This is the  $r$ -th round ( $r = i + 1$ )
     $Y_i = \text{SubNib}(X_i)$ 
     $Z_i = \text{AddConst}(Y_i)$ 
     $U_i = \text{PosPerm}(Z_i)$ 
     $W_i = \text{MixColumn}(U_i)$ 
     $X_{i+1} = W_i \oplus RK_{i+1}$  ▷ Round key addition
end for
 $Y_{N-1} = \text{SubNib}(X_i)$  ▷ The last round
 $X_N = Y_{N-1} \oplus RK_N$ 
 $C = X_N$ 

```

SubNib. Substitution referred to as S-box is the only nonlinear operation based on nibbles, which divides 128-bit state X into 32 nibbles $X = \{x_0, x_1, \dots, x_{31}\}$. These nibbles

are substituted for other nibble values according to the table S-box, and the output state is obtained Y , where $Y = \mathbf{SubNib}(X) = \{S(x_0), S(x_1), \dots, S(x_{31})\}$. The S-box is shown in Table 2.

Table 2: The (low-latency) S-box S .

Input	0x0	0x1	0x2	0x3	0x4	0x5	0x6	0x7	0x8	0x9	0xa	0xb	0xc	0xd	0xe	0xf
Output	0x8	0x0	0x1	0x5	0xc	0x7	0x4	0x6	0x2	0xa	0x3	0xd	0xe	0xf	0xb	0x9

AddConst. The first two columns are XORed with a 32-bit constant $c = \{c_0, c_1, c_2, c_3, c_4, c_5, c_6, c_7\}$ which is constructed based on 6-bit LFSR, and the feedback polynomial is $(a_0, a_1, a_2, a_3, a_4, a_5) \rightarrow (a_5 \oplus a_4 \oplus 1, a_0, a_1, a_2, a_3, a_4)$. The round constants corresponding to (c_0, c_1) are shown in Table 3. For uLBC-128, $(c_2, c_3, c_4, c_5) = 0xc5c5$, uLBC-256, $(c_2, c_3, c_4, c_5) = 0x3c3c$, uLBC-384t, $(c_2, c_3, c_4, c_5) = 0x5a5a$. It is used to distinguish the algorithm versions corresponding to different versions. The round constant corresponding to (c_6, c_7) consists of the decimal part of π , as shown in Table 4. $Z = \mathbf{AddConst}(Y)$, where $z_i = y_i \oplus c_i$, $i = 0, 1, 2, 3$, and $z_i = y_i$, $i = 4, 5, \dots, 31$.

Table 3: The first two nibbles of round constant addition.

Round	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
(c_0, c_1)	0x80	0xc0	0xe0	0xf0	0xf8	0x7c	0xbc	0xdc	0xec	0xf4	0x78	0x3c	0x9c	0xcc	0xe4	0x70
Round	16	17	18	19	20	21	22	23	24	25	26	27	28	29	30	31
(c_0, c_1)	0xb8	0x5c	0xac	0xd4	0x68	0x34	0x18	0xc0	0x84	0x40	0xa0	0xd0	0xe8	0x74	0x38	0x1c

Table 4: The last two bytes of round constant addition.

Round	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
(c_6, c_7)	0xa0	0xac	0x93	0x29	0xac	0x4b	0xc9	0x91	0xc2	0x31	0x32	0x19	0xc1	0x93	0xca	0x81
Round	16	17	18	19	20	21	22	23	24	25	26	27	28	29	30	31
(c_6, c_7)	0x44	0x20	0xcb	0x8b	0x49	0xcc	0x9b	0xa8	0x82	0xc1	0x04	0xba	0x4a	0x22	0xc9	0x18

PosPerm. A cell-wise permutation P_S is applied to the state. $U = \mathbf{PosPerm}(Z) = \{u_0, u_1, \dots, u_{31}\}$, where $u_i = z_{P_S(i)}$, $i = 0, 1, \dots, 31$. The permutation P_S is shown in Table 5.

MixColumn. Multiply each column of the intermediate state matrix with a 4×4 binary matrix M , and the matrix M is shown below:

$$M = \begin{pmatrix} 0 & 1 & 1 & 1 \\ 1 & 0 & 1 & 1 \\ 1 & 1 & 0 & 1 \\ 1 & 1 & 1 & 0 \end{pmatrix},$$

where the matrix $M^{-1} = M$. $W = \{w_0, w_1, \dots, w_{31}\} = \mathbf{MixColumn}(U)$, that is:

$$\begin{pmatrix} w_{4j} \\ w_{4j+1} \\ w_{4j+2} \\ w_{4j+3} \end{pmatrix} = M \begin{pmatrix} u_{4j} \\ u_{4j+1} \\ u_{4j+2} \\ u_{4j+3} \end{pmatrix}, \quad j = 0, 1, \dots, 7.$$

If the cells are not all equal to 0 for the input $\{u_{4j}, u_{4j+1}, u_{4j+2}, u_{4j+3}\}$, there are at least four non-zero cells among the input and output cells. Hence, the branch number is 4.

AddRK. The state W after the column mixing operation and the round key RK_r ($r = 1, 2, \dots, N$) are bitwise exclusive-ored to obtain the output state.

Table 5: The cell permutation P_S .

Input x								→	Output $P_S(x)$							
0	4	8	12	16	20	24	28		0	4	8	12	20	16	28	24
1	5	9	13	17	21	25	29		25	1	29	9	5	13	17	21
2	6	10	14	18	22	26	30		18	30	22	26	10	2	6	14
3	7	11	15	19	23	27	31		15	19	7	23	31	27	3	11

2.3 Decryption

The decryption algorithm is the inverse operation of the encryption algorithm, involving the execution of N -round iterative operations. These operations include the addition of the round key, the inverse of column mixing (MixColumn^{-1}), the inverse of cell permutation (PosPerm^{-1}), and the inverse of the S-box substitute (SubNib^{-1}). The decryption ends with an exclusive OR operation with the key RK_0 , producing the plaintext. The pseudocode for the decryption algorithm is presented in Algorithm 2.

Algorithm 2 The Decryption of uLBC.

Input: ciphertext: C , round key: $RK_0, \dots, RK_N$

Output: plaintext: P

$X_N = C$

$Y_{N-1} = X_N \oplus RK_N$

▷ The last round

$X_{N-1} = \text{SubNib}^{-1}(Y_{N-1})$

for $i = N - 2$ **to** 0 **do**

▷ This is the r -th round ($r = N - i$)

$W_i = X_{i+1} \oplus RK_{i+1}$

▷ Round key addition

$U_i = \text{MixColumn}^{-1}(W_i)$

$Z_i = \text{PosPerm}^{-1}(U_i)$

$Y_i = \text{AddConst}(Z_i)$

$X_i = \text{SubNib}^{-1}(Y_i)$

end for

$P = X_0 \oplus RK_0$

▷ XORed with the whitening key

AddRK. During decryption, the initial state X_N is ciphertext C . In the decryption process of the r round ($r = 1, 2, \dots, N$), the state X_{N-r+1} and the round key RK_{N-r+1} are exclusive-ored, and the last state and key RK_0 are exclusive-ored to obtain the output plaintext P .

MixColumn⁻¹ Same as **MixColumn**, because of $M^{-1} = M$, multiply each column of the intermediate state matrix with the 4×4 binary matrix M .

PosPerm⁻¹ In the state of 32 nibbles, cell permutation is performed. The cell permutation position of these 32 units is in Table 6.

Table 6: The cell permutation P_S^{-1} .

Input x								→	Output $P_S^{-1}(x)$							
0	4	8	12	16	20	24	28		0	4	8	12	20	16	28	24
1	5	9	13	17	21	25	29		5	17	13	21	25	29	1	9
2	6	10	14	18	22	26	30		22	26	18	30	2	10	14	6
3	7	11	15	19	23	27	31		27	11	31	3	7	15	23	19

188 **AddConst** The first two columns of the state and 32-bit constant $c = \{c_0, c_1, c_2, c_3,$
 189 $c_4, c_5, c_6, c_7\}$ are XORed, and the constant value is in the opposite order to the encryption.

190 **SubNib**⁻¹ The S-box substitution inversion is also a non-linear operation performed on
 191 nibbles. The 32 nibbles are substituted one by one through the inverse S-box table to
 192 obtain the output state $X = \mathbf{SubNib}^{-1}(Y)$.

193 2.4 Key Schedule

194 The key scheduling algorithm incorporates cell permutation to reduce area and improve
 195 efficiency. The key schedule of uLBC-256 exhibits slightly greater complexity compared
 196 to that of uLBC-128. We use the functions f_1 and f_2 for the nibble, which are based on
 197 LFSR construction with a cycle of 15.

198 For uLBC-128, the key length equals the block length. Its key schedule algorithm
 199 uses the function F and key K to generate round keys RK_i , $i = 0, 1, \dots, N$, that is,
 200 $RK_0 = K$, $RK_i = F(RK_{i-1})$, $i = 1, \dots, N$.

201 The function F is a cell permutation. The 128-bit state is divided into 32 nibbles
 202 (4-bit cell). The nibble-based matrix has four rows and eight columns. The permutation
 203 of the cell position is shown in Table 7. The input and output of F function are $X =$
 204 $\{x_0, x_1, \dots, x_{31}\}$ and $Y = \{y_0, y_1, \dots, y_{31}\}$ respectively.

The pseudocode of the round function $Y = F(X)$ is $y_i = x_{P_K[i]}$, $i = 0, 1, \dots, 31$.

Table 7: cell permutation table P_K .

Input i								$\rightarrow$	Output $P_K[i]$							
0	4	8	12	16	20	24	28		8	9	20	18	4	10	21	27
1	5	9	13	17	21	25	29		24	31	19	2	15	29	16	13
2	6	10	14	18	22	26	30		28	14	26	6	11	12	7	22
3	7	11	15	19	23	27	31		23	17	5	30	25	3	0	1

205 For uLBC-256, the key length is twice the block length. The key scheduling algorithm
 206 initially divides the $2n$ -bit master key K into two n -bit round keys $K = K_0 || K_1$. Subse-
 207 quently, it employs the F function and the f_1 function to generate the round keys. The
 208 f_1 function involves nibble transformation, while the F function remains consistent with
 209 uLBC-128. Specifically, $RK_0 = K_0 \oplus K_1$, $RK_i = F^i(K_0 \oplus f_1^i(K_1))$, $i = 1, \dots, N - 1$. f_1
 210 is constructed on the basis of a 4-bit LFSR with a feedback polynomial $(a_0, a_1, a_2, a_3) \rightarrow$
 211 $(a_0 \oplus a_3, a_0, a_1, a_2)$. The corresponding inputs and outputs are detailed in Table 8.

Table 8: Nibble substitution table for f_1 .

Input	0x0	0x1	0x2	0x3	0x4	0x5	0x6	0x7	0x8	0x9	0xa	0xb	0xc	0xd	0xe	0xf
Output	0x0	0x8	0x1	0x9	0x2	0xa	0x3	0xb	0xc	0x4	0xd	0x5	0xe	0x6	0xf	0x7

For uLBC-384t, we regard the key and tweak as a unified tweakkey, following the
 TWEAKEY schedule [JNP14]. The length of key and tweak can be flexibly chosen, but
 the length of key must be at least as large as the block size, i.e., 128-bit. The tweakkey
 schedule also first divides the $3n$ length tweakkey TK into three n round tweakkeys, as
 $TK = TK_0 || TK_1 || TK_2$. Subsequently, the F function is applied as a cell permutation,
 similar to uLBC-128 in each round, and the f_1 and f_2 functions are utilized for the nibble
 transformations. The specifics are detailed below:

$$RK_0 = TK_0 \oplus TK_1, RK_i = F^i(TK_0 \oplus f_1^i(TK_1) \oplus f_2^i(TK_2)), i = 1, \dots, N - 1.$$

213 Similar to f_1 , the f_2 is also constructed based on 4-bit LFSR, with the feedback
 214 polynomial $(a_0, a_1, a_2, a_3) \rightarrow (a_1, a_2, a_3, a_0 \oplus a_1)$. The corresponding inputs and outputs
 215 are in Table 9.

Table 9: Nibble substitution table for f_2 .

Input	0x0	0x1	0x2	0x3	0x4	0x5	0x6	0x7	0x8	0x9	0xa	0xb	0xc	0xd	0xe	0xf
Output	0x0	0x2	0x4	0x6	0x9	0xb	0xd	0xf	0x1	0x3	0x5	0x7	0x8	0xa	0xc	0xe

3 Design Decisions

3.1 Main Ideas and Strategies of Design

This paper aims to design a family of low-latency block ciphers while providing strong security arguments. AES is the most widely deployed block cipher, known for its bytes-based design framework with high throughput and strong security. AES, with a strong diffusion layer comprising ShiftRows and MixColumns, has been proven resistant to various attacks such as differential, impossible differential, and linear attacks. In AES, the MixColumns operation employs an MDS matrix with a branch number of 5. To meet minimal area and energy requirements, the block cipher Midori128 utilizes an AES-like design. It incorporates a low-latency 8-bit S-box composed of two 4-bit S-boxes. Instead of using ShiftRows, it utilizes a byte permutation, and it replaces the original MDS matrix of AES with an almost 4×4 MDS matrix having a branch number of 4. To obtain low-latency, the PRF of Orthros XORs two 128-bit outputs of permutations, each permutation employing bit permutation and nibble permutations, along with a 4×4 almost MDS matrix, to facilitate strong diffusion. However, Orthros is solely used as a pseudo-random function. The cipher employs bit permutation, making it challenging to estimate the lower bound of the active S-box count to derive the security arguments.

In the context of proving the security of a low-latency block cipher, we focus on components such as 4-bit S-boxes (or 8-bit S-boxes), 32-nibble permutations (or 16-byte permutations), and low-latency MDS matrices (or almost MDS matrices). Among these options, we identify a round function with a 4-bit S-box as the confusion function, a nibble permutation, and an almost MDS matrix as the diffusion function. This combination achieves low latency and low area, making it suitable for efficient hardware implementations. The choice of a 4-bit S-box is friendly for AVX instructions to enhance software implementations using parallel modes, such as CTR and GCM.

For the diffusion function, the selection of nibble permutation has worked in the bounds of the number of active S-boxes and the length of an impossible differential. We select nibble permutation carefully to provide stronger bounds on the number of active S-boxes. Although several design choices have been borrowed from existing ciphers, we give some new methods to optimize for low latency and lightweight.

Furthermore, we apply 4-bit S-boxes to optimize the depth of the circuit. A new method for constructing low-latency coordinate functions is proposed, which is more consistent with the implementation. Using a proper combination of coordinate functions, a large number of low-latency S-boxes are constructed.

Based on the above research, we propose a 128-bit block cipher that supports both 128-bit and 256-bit keys. To achieve lightweight properties, we apply a linear key schedule based on nibble permutation for 128-bit keys. This schedule effectively generates key diffusion, providing resistance against related key attacks and weak key attacks. The nibble permutation is employed to extend the TWEAKEY frame of SKINNY[BJK⁺16] for the 256-bit key configuration. The Tweakey frame can support longer tweaks and keys. Therefore, we present a 384-bit tweakable block cipher that accommodates a 128-bit tweak and keys ranging from 128 bits to 256 bits. The Tweakey framework treats the key and tweak interchangeably. Hence, it is necessary to provide related-key attack resistance in Tweakey versions.

3.2 The Design of Round Function

We employ a 128-bit block size, which is a commonly used size to optimize encryption and decryption efficiency. Our cipher utilizes the Substitution-Permutation Network (SPN) structure, with the round function consisting of both a confusion layer and a diffusion layer. In this section, we focus on selecting the appropriate components. The 4-bit S-boxes are widely used in low-latency ciphers such as Midori, QARMA, MINTIS, etc.

Here, We take 4-bit S-boxes as the basic cell to design a 128-bit block cipher, which means that the 128-bit block is divided into 32 4-bit S-boxes. In addition, a linear layer is constructed for the 32 nibbles, which are represented by a 4×8 matrix. Here, the nibble permutation plays an important role in determining the number of active S-boxes. Here, we use a reliable nibble permutation **PosPerm** to compare the differential diffusion of the MDS matrix and the almost MDS matrix. For the diffusion layer, we compare the efficiency of the following two cases:

Case I We use nibble permutation (**PosPerm**) and almost MDS matrix M , which is shown in Subsection 2.2.

Case II We use nibble permutation (**PosPerm**) and the low-latency MDS matrix M_4 . The modular polynomial is $x^4 + x + 1$, with values 0x13. This MDS is given in the [LS16]. There are 4 XOR operations for each output bit.

$$M_4 = \begin{pmatrix} 1 & 1 & 4 & 9 \\ 1 & 4 & 9 & 1 \\ 4 & 9 & 1 & 1 \\ 9 & 1 & 1 & 4 \end{pmatrix},$$

For cryptanalysis, the number of 4-bit active S-boxes is the main criterion for evaluation against differential and linear attacks. We analyze the minimized number of active S-boxes in cryptanalysis and the rounds of the full diffusion, which are listed in Table 10. Midori128 incorporates a low-latency 8-bit S-box composed of two 4-bit S-boxes which are the basic units. It has 41 active S-boxes for 9-round differential characteristics, 62 active S-boxes for 12-round differential characteristics, and 67 active S-boxes for 13-round differential characteristics. However, there are 68 active S-boxes for 9-round differential characteristics in **Case I** and 74 active S-boxes for 9-round differential characteristics in **Case II**, both more than 64 active S-boxes. Hence, there are at most 8-round differential characteristics for the two cases. It is evident that the latter two have stronger diffusion to resist differential attacks.

Now we consider the low latency of the hardware implementation. The diffusion layer has the same XOR computation for Midori128 and **Case I**. Here we compare the implementation for **Case I** and **Case II**. There are 4-round full diffusion for **Case I** and 3-round full diffusion for **Case II**. We compare the latency for 18-round **Case I** and 16-round **Case II**, and the first case has a little advantage over the second case. Hence, we use the round function of **Case I**. Subsequently, we introduce the design of the diffusion layer and the low-latency S-boxes to improve the throughput.

Table 10: The number of active S-boxes of differential for the 32-nibble block.

linear layer	round number of full diffusion	rounds								
		1	2	3	4	5	6	7	8	9
Midori128	3	1	4	7	16	23	30	35	38	41
Case I	4	1	4	7	16	25	40	52	60	68
Case II	3	1	5	9	25	41	55	58	62	74

3.3 The Design of Diffusion Layer

We use Case I as the round function, which is composed of a nibble permutation and a 4×4 almost MDS binary matrices. The almost MDS binary matrices have been used in the design of PRINCE, Midori, MANTIS, Orthros, etc. However, the 4×4 almost MDS binary matrices have a branch number of 4 less than the MDS matrices, resulting in slow diffusion. Banik *et al.* utilize bit and nibble permutations in a hybrid manner to improve the diffusion speed and to increase active S-boxes in each round [BIL⁺21]. However, the bit permutation is not suitable for software implementation, which usually makes the implementation speed slow. Besides, the search space for bit permutation is too large to give accurate estimates of the active S-box number in the differential or linear path. Here, we applied the nibble permutation. There are $32! \approx 2^{117}$ permutations for 32 nibbles. It is impossible to traverse. Banik *et al.* give some analysis to select good nibble permutation, such as Condition 1.

Condition 1 (Condition 3 [BIL⁺21]). *For each column $(u_{4i}, u_{4i+1}, u_{4i+2}, u_{4i+3})$ in the 4×8 array, after applying the nibble permutation, they will be mapped to four nibble-cells in different columns.*

Banik *et al.* randomly choose 7,000 nibble permutations satisfying condition 1, and then compute the lower bound of the number of active S-boxes after 5, 6, 7, and 8 rounds for these nibble permutations with the MILP model. Among them, they find three nibble permutations that can achieve 60 active S-boxes over 8 rounds.

The MILP model is a time-consuming operation. Here we give some more conditions to find a good nibble permutation efficiently.

Condition 2. *For each column $(u_{4i}, u_{4i+1}, u_{4i+2}, u_{4i+3})$ in the 4×8 array, after applying the nibble permutation twice, they will be mapped to four nibble-cells in different columns.*

This condition is used to optimize the bound of active boxes in the differential path.

Condition 3. *For each active input of the S box in the first round, it will generate 9 active S-box after two rounds, which will be mapped to all 8 columns after the **PosPerm**.*

In the case of an active box transformation, $1 \rightarrow 3 \rightarrow 9 \rightarrow (18 \sim 27) \rightarrow 32$. Hence, the 4-round diffusion needs to be taken into account. We give the following conditions.

Condition 4. *For each active S-box input in the first round, it will propagate the difference to all cells in the fourth round, i.e., there are at least 2 active S-box in each column before **MixColumn** in the fourth round.*

The Condition 4 is used to ensure the full diffusion, which is used to limit the length of an impossible differential. Because **MixColumn** can mix the column, we choose the row permutation independently for different rows satisfying all the conditions above. There are exactly $8! = 40320$ row permutations. We searched all these cases, and there are about 695520 permutations inconsistent with all 4 conditions. Then we search the differential path by estimating the active S-boxes for 6,7,8,9 rounds. We found that when the number of active S-boxes is 40 for 6-round uLBC with a selected nibble permutation, it is hard to find a 9-round differential with high probability. The permutation found is row-independent, therefore, we can implement the round function with the AVX instructions efficiently.

Table 11 shows a comparison of the lower bound of the number of active S-boxes for Midori128, Branch1 and Branch2 of Orthros, and uLBC-128. Compared with Branch1/2 of Orthros, our nibble permutations guarantee a much larger number of active S-boxes for 6 rounds and 7 rounds. Our nibble permutation causes a stronger diffusion at the same time, which leads to an 8-round impossible differential. However, we find many 9-round impossible differentials for Branch1/2 of Orthros, respectively, seen in Table 12 and

3.4.1 Logic Gates and Their Latency

To evaluate the latency of the coordinate functions, the most popular metric is depth, which assesses the path delay of the S-box by considering the sequential performance of the logic gates involved [BBI⁺15, BGLS19, Ras22]. Leander *et al.* highlighted the limitations of the depth metric by analyzing the latency of logic gates by calculating the Fan-in-to-Latency Ratio (FLR) for each gate and observing that NAND and OAI gates get the highest FLR scores among all logic cells tested [LMMR21].

When building 4-bit S-boxes, we found that NAND does have the lowest latency but some logic gates also perform well in terms of latency, which is ignored in [LMMR21]. The traditional depth metric is based on the number of logic gates to sequentially proceed in the operation and only considers gates with fan-in of no more than 2.

In this paper, we introduce *logical effort* as the metric instead of depth and give logical efforts for some general logic gates. The logical effort method, introduced by Sutherland *et al.* [SSH99], offers a technique to design MOS circuits to achieve high speeds. This approach simplifies MOS circuits as networks of resistance and capacitance for circuit analysis. The delay of a logic gate d is the summation of the parasitic delay p and the effort delay f , defined as $d = f + p$. The effort delay f is the product of the logical effort g and the electrical effort h , given by $f = gh$. The electrical effort h characterizes the cost of driving capacitive loads, commonly referred to as fanout by many CMOS designers. For simplicity, we assume it is fixed to 1 in the subsequent discussion. Logical effort g describes the computational cost attributable to the circuit topology. The logical effort of the inverter is defined as 1 delay unit, thus all logical efforts are gauged relative to the delay of a basic inverter. Since the logical effort of a gate can be computed by assessing how much additional input capacitance a gate presents to deliver the same output current as an inverter, the logical efforts of other gates are derived based on the ratio of input capacitances. The parasitic delay p of a logic gate is predetermined, with the primary contribution being the capacitance of the source/drain regions of the transistors that drive the gate output. The parasitic delay is a multiple of the parasitic delay of the inverter, which is conventionally set to 1 delay unit. Table 14 presents assumed values of the logical effort and the parasitic delay for various gates. This table was simplified based on [SSH99]. As [SSH99] is used mainly for circuit design, it takes into account various factors more carefully, such as the influence of fanouts, etc. Therefore, simplification is reasonable.

Table 14: Values of delay factors for some logical gates.

Gate	Fan-in	Logical effort (g)	Parasitic delay (p)
INV	1	1.0	1.0
NAND2	2	1.33	2.0
NOR2	2	1.67	2.0
NAND3	3	1.67	3.0
NOR3	3	2.33	3.0
OAI21	3	2.0	3.0
AOI21	3	2.0	3.0
NAND4	4	2.0	4.0
NOR4	4	3.0	4.0
OAI22	4	2.0	4.0
AOI22	4	2.0	4.0

In logic networks, the path logical effort G is the product of the logical effort of each logic gate along the path, defined as $G = \prod g_i$. Further, the path effort F is computed as the product of G and H , i.e., $F = G \cdot H$. Here, $H = \prod h_i$ represents the effort of the individual stages along the path. In practice, it is often possible to expand the circuit

and utilize additional area to decrease the number of fan-ins and fan-outs, resulting in $H = 1$. Consequently, the path effort simplifies to $F = G$, as the individual stage efforts cancel out. The path parasitic latency denoted as P , is defined as the sum of the parasitic latencies $P = \sum p_i$. For an N -stage logic network, the path delay can be computed by

$$\mathcal{D} = N \cdot F^{\frac{1}{N}} + P. \quad (1)$$

The path delay of Boolean functions is the maximum path delay of each input wire. We give an example to show how to compute the path delay.

Example 1. The path delay of a 3-bit S-box. The S-box is

$$\begin{aligned} y_1 &= \text{INV}(\text{NAND2}(\text{NAND2}(x_1, x_2), \text{NAND2}(\overline{x_1}, x_3))), \\ y_2 &= \text{NAND2}(\text{NAND2}(x_2, x_1), \text{NAND2}(\overline{x_2}, x_1)), \\ y_3 &= \text{NAND2}(\text{NAND2}(x_3, x_1), \text{NAND2}(\overline{x_3}, x_2)). \end{aligned}$$

For a 3-bit S-box, there are 3 input values (i.e., x_1, x_2 , and x_3) and 12 input wires. We number them in sequence from 1 to 12, where the delays of index 1, 2, and 4 are the same, the delays of index 5, 6, 8, 9, 10, and 12 are the same, and the delays of index 7 and 11 are the same. Thus we can obtain

$$\begin{aligned} \mathcal{D}_{1,2,4} &= 3 \cdot (1 \cdot 1.33 \cdot 1.33)^{1/3} + (1 + 2 + 2) = 8.63, \\ \mathcal{D}_3 &= 4 \cdot (1 \cdot 1.33 \cdot 1.33 \cdot 1)^{1/4} + (1 + 2 + 2 + 1) = 10.61, \\ \mathcal{D}_{5,6,8,9,10,12} &= 2 \cdot (1.33 \cdot 1.33)^{1/2} + (2 + 2) = 6.66, \\ \mathcal{D}_{7,11} &= 3 \cdot (1.33 \cdot 1.33 \cdot 1)^{1/3} + (2 + 2 + 1) = 8.63. \end{aligned}$$

Thus the path delay of this S-box is the maximum path delay among all Boolean functions, i.e., $\mathcal{D} = \mathcal{D}_3 = 10.61$.

3.4.2 S-box Searching

Using the path delay, the first step is to find several coordinate functions for 4-bit S-boxes with low latency. We select some common logic gates, as shown in Table 14, and construct Boolean functions based on the delays of all the logic gates along the path of the Boolean function. Table 14 lists values of delay factors for all the logic gates we selected. According to equation (1), the delay of a Boolean function $\mathcal{D}$ is derived as $\mathcal{D} = N \cdot F^{\frac{1}{N}} + P$.

In our search, to address the challenge of path delays, we iterate based on the depths of logic gates. We assign weights to the depths of different gates, assuming that the depths of INV, NAND2/NOR2, NAND3/NOR3/OAI21/AOI21, and NAND4/NOR4/OAI22/AOI22 gates are weighted as 0.5, 1, 2 and 2, respectively. The BUF gate $y = x_0$, $y = x_1$, $y = x_2$, $y = x_3$, and the INV gate $y = \neg x_0$, $y = \neg x_1$, $y = \neg x_2$, $y = \neg x_3$, are set to have factors:

	Depth	Path logical effort	Path parasitic delay
BUF	0	1.0	0
INV	0.5	1.0	1.0

When searching Boolean functions whose depths are s , we apply INV, NAND2/NOR2, NAND3/NOR3/OAI21/AOI21, and NAND4/NOR4/OAI22/AOI22 to the Boolean functions with depth $s - 0.5$, $s - 1$, $s - 2$ and $s - 2$ respectively, and check whether the Boolean functions satisfy our requirements: 1) The path delay is no more than 10. 2) The depth is no more than 4.5. 3) The output is 0-1 balanced. 4) The function expression contains all 4 bits. 5) The algebraic degree is greater than 1. There is no special reason for the first and second conditions since some Boolean functions can already be found within these

parameters. We can also set them based on the path delay of some known S-boxes, such as the SPEEDY S-box [LMMR21]. After that, we use these Boolean functions to construct 4-bit S-boxes by the method of [Ras22]. Note that to reduce the search space, we require that the coordinate functions of the S-box be sorted by truth table during the search. Our selected S-boxes are listed below:

Table 15: The structure of our low-latency S-box. S_i denotes the i th coordinate functions.

Function	Structure	Delay $\mathcal{D}$
S_0	$\text{NAND2}(\text{NAND3}(\text{INV}(x_0), \text{INV}(x_2), \text{INV}(x_3)), \text{OAI21}(x_1, x_3, x_0))$	9.8
S_1	$\text{NAND2}(\text{NAND3}(\text{INV}(x_0), x_1, x_3), \text{OAI21}(\text{INV}(x_2), \text{INV}(x_3), x_0))$	10.1
S_2	$\text{NAND2}(\text{NAND3}(\text{INV}(x_2), x_3, x_1), \text{OAI21}(x_0, \text{INV}(x_1), x_2))$	10.1
S_3	$\text{NAND2}(\text{NAND3}(\text{INV}(x_1), x_3, x_2), \text{OAI21}(\text{INV}(x_0), \text{INV}(x_2), x_1))$	10.1

We find a few S-Boxes that fit the conditions and come up with the one that performs best in Table 15. The DDT and LAT of the S-box S we used are shown in Figure 2.

Table 16 lists the depths, delays, and practical latency of S-boxes in some low-latency block ciphers. Our metric uses finer granularity than depth, for example, the S-boxes with depth 3.5 in Table 16 are further divided into two categories $\mathcal{D} = 10$ and $\mathcal{D} = 11$, and the S-boxes with depth 4 are further divided into categories $\mathcal{D} = 12$ and $\mathcal{D} = 13$. We observed that the latency of the S-boxes is basically in line with our estimates. Generally, S-boxes with high $\mathcal{D}$ values take an advantage in latency, while there are a few exceptions, such as Midori-Sb1, which has lower latency than those S-boxes with $\mathcal{D} = 11$. Besides, in the experiments, the S-box of QARMA- σ_2 has lower latency than the inverse operation (QARMA- σ_2_Inv), but the depth value of QARMA- σ_2 is greater than that of QARMA- σ_2_Inv and did not match the measured values. When using our metric, both S-boxes have the same $\mathcal{D}$ values, which is at least consistent with the experimental results of 45nm.

Table 16: Properties of some low latency S-boxes

S-box	Depth	$\mathcal{D}$	Latency	
			15nm(ps)	45nm(ns)
Orthros	3.5	10	4.95496	0.09573
Orthros_Inv	4	13	9.04691	0.14699
Midori-Sb0	3.5	10	5.26489	0.10603
Midori-Sb1	4	12	7.68204	0.1066
QARMA- σ_0	3.5	11	4.95496	0.11188
QARMA- σ_1	4	12	7.27624	0.1259
QARMA- σ_2	4.5	13	7.8453	0.14516
QARMA- σ_2_Inv	4	13	9.04691	0.14699
Our S-box	3.5	10	4.77033	0.09286

3.5 The Design of Key Schedule and Constants

Block cipher uLBC supports a 128-bit block, and the key length supports 128 bits and 256 bits. We also design a version of the 128-bit block and 384-bit tweakey. Although a little complex key schedule does not increase encryption latency, we apply a simple key schedule by nibble permutation for the 128-bit key to reduce the cost of implementation.

For a 256-bit key, we make use of the nibble permutation to extend the Tweakey frame of SKINNY [JNP14] with tweaked size of twice the block length. The 256-bit key is divided into two equal parts. The nibble permutation F is used for the two parts independently. An S-box substitute f_1 is used in each nibble of the second part for each round. f_1 is constructed by a 4-bit LFSR with a period of 15. Hence, the masterkey can be computed

by any two consecutive round subkeys. For the 384-bit tweakey, the key schedule is similar to the 256-bit key. The only difference is the 384-bit tweakey is divided into three parts, and the S-box substitute f_1 and f_2 are used in each nibble of the last two parts for each round.

For the nibble permutation F in uLBC-128, uLBC-256, and uLBC-384t, we choose the same nibble permutation to reduce implementation costs. Hence, we add some conditions on the nibble permutation in the key schedule to remove some iteration paths in the related-key setting to sieve good nibble permutation. Then we check the nibble permutation by the number of active S-boxes in the related-key setting.

Many lightweight block ciphers apply a very simple key schedule in which the round keys only differ by the addition of a round-specific constant, which is vulnerable to invariant subspace attacks. Beierle *et al.* [BCLR17] analyzed the resistance of some ciphers against invariant attacks and gave a method to find round constants that guarantee the resistance to all types of invariant attacks. Following that, we choose the round constants. There are three parts: one is used to distinguish four versions of uLBC; another part is constructed by the LFSR with a period of 63 to resist the invariant attacks; the last part is used as random numbers.

4 Security Analysis

This section provides cryptanalysis of the block cipher uLBC, including various attack techniques such as differential attack, boomerang attack, linear analysis, impossible differential analysis, integral analysis, and related-key analysis. The summary of the analysis results are shown in Table 19.

4.1 Differential/Linear Cryptanalysis

4.1.1 Basic Properties and Nibble-Level Bounds

The differential distribution table (DDT) and linear approximation table (LAT) of the S-box are shown in Figure 2. As we can see, the highest differential/linear probability for each S-box is 2^{-2} .

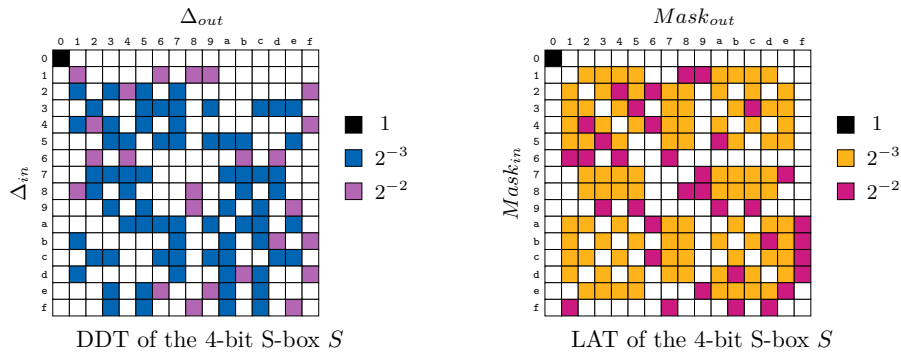


Figure 2: Differential/Linear properties of the S-Box S .

To argue for the resistance of uLBC against differential and linear attacks, we computed lower bounds on the minimal number of active S-boxes. Based on bit-level differential propagation, we consider all possible differential characters of S-box and build an SAT automatic search model for solving the minimum S-boxes. We get the results for uLBC-128

with 1-9 rounds, the results are shown in Table 17. Because of the equivalence between differential and linear trails, the result for the linear attack is the same. For verification, we present our differential trails with 3 to 7 rounds in Table 22, Appendix B.

Table 17: Estimation of the number of active S-boxes.

Round	1	2	3	4	5	6	7	8	9
Number	1	4	7	16	25	40	52	60	68

We prove that the minimum number of active S-boxes in 9-round uLBC-128 is at least 68, and the maximum differential characteristics probability will not be higher than 2^{-136} . Meanwhile, we consider the probability of clustering of differential trails. We search for clusterings with our SAT automatic search tools, the searching phase is carried out in two steps. First, we search for the differential trail with the maximum probability. Then, we fix the input and output differentials equal to the trail, and add constraints to ensure that the probability is greater than 2^{-10} times of the maximum probability, while searching for all eligible trails. We continue this process until we have an adequate number of trials in the clustering. Finally, we evaluate the actual probability of the input-output differential characteristics. The clustering with the highest probability for 8-round uLBC-128 is about $2^{113.1}$, and $2^{128.6}$ for 9-round. Therefore, it is expected that the algorithm does not have an effective differential trail of 9 rounds, and 4 rounds of the algorithm can spread to all active boxes. Therefore, the differential (or linear) attack potentially can analyze 16 rounds of uLBC-128 at most.

4.1.2 Key Recovery

The 8-round distinguisher shown in Table 22 leads to 24 active S-boxes at one round before, and 28 active S-boxes at the round after, such that, only 1-round extension can be appended to before and after the differential/linear distinguisher to launch a valid attack for 128-bit version uLBC-128. The round of attack is bounded by 10 rounds, although some techniques can be used to extend the attack by one or two rounds, there is still 8-round redundancy to protect the security.

And for 256-bit key version uLBC-256, two rounds can be appended before and after the 8-round distinguisher (after two-round propagation in forward/backward direction, the s-boxes are all active in the state), the expected number of attack rounds is $2 + 8 + 2 = 12$.

Even in the ideal case, there is a valid 8-round differential/linear distinguisher with only one active nibble at the beginning and one active nibble in the output state, the full diffusion will occur through four rounds before and after the distinguisher, for uLBC-128, the upper bound of the number of round we possible attack is $4 + 8 + 4 = 16$. For uLBC-256, one more round could be padded at the beginning and the end, the attack round is bounded by $5 + 8 + 5 = 18$. And for uLBC-384-t, we possibly attack at most $6 + 8 + 6 = 20$ rounds. There are also more than two rounds redundancy.

4.1.3 Differential-Linear Cryptanalysis

Differential-linear cryptanalysis [LH94] is a cryptanalytic method that combines aspects of differential cryptanalysis and linear cryptanalysis to analyze block ciphers. The method divides the cipher E into two parts, $E = E_1 \circ E_0$, where there exists a strong truncated differential for the first part E_0 with probability $p = \Pr[\alpha \xrightarrow{E_0} \beta]$ and there exists a strongly biased linear approximation with bias ϵ for the second part E_1 , such that $\Pr[\Gamma_0 \xrightarrow{E_1} \Gamma_1] = 1/2 + \epsilon$. The bias of differential-linear distinguisher is $\epsilon_{\alpha, \Gamma_1} = \Pr[\Gamma_1 \cdot (E(x \oplus \alpha) \oplus E(x)) = 0] - 1/2 = 4p\epsilon^2$, where $\Pr[\Gamma_0 \cdot \beta = 0] = 1$. The differential-linear distinguisher is effective, when $\epsilon_{\alpha, \Gamma_1} > 2^{-n/2}$ for n -bit block size. Based on the number of active S-

boxes for differential and linear analysis as discussed in Section 4.1.1, the differential-linear distinguisher can apply to a maximum of 8 rounds.

4.2 Boomerang-type Attacks

The boomerang-type attacks treats a block cipher E as the composition of two sub-ciphers E_0 and E_1 , where have two differentials $\alpha \xrightarrow{E_0} \beta$ and $\gamma \xrightarrow{E_1} \delta$ with probabilities p and q respectively. Then the probability of a boomerang distinguisher is estimated by $Pr[E^{-1}(E(x) \oplus \delta) \oplus E^{-1}(E(x \oplus \alpha) \oplus \delta) = \alpha] = p^2 q^2$. The probability of $p = Pr[\alpha \xrightarrow{E_0} \beta]$ and $q = Pr[\gamma \xrightarrow{E_1} \delta]$ is mainly bouned by the active S-boxes in the differential. According to Table 17, we estimate the longest boomerang-type distinguisher will not exceed 8-rounds $(3 + 1 + 4)$. In fact, we test the boomerang probability with the sandwich attack [DKS10] which introduces a sub-cipher E_m in the middle, utilizing our 4-round differentials with the minimal number of active S-boxes, and found there is no boomerang distinguisher with $(4+1+4)$ -form can return in a valid probability.

4.3 Impossible Differential Analysis

Impossible differential attack [BBS99] finds two internal state differences Δ_f , Δ_r such that Δ_f will never propagated to Δ_r .

The full diffusion will occur through four rounds of uLBC, so we can construct 8-round impossible differential trails. In Table 18, we show an 8-round impossible differential. The zero-correlation trail of the cipher is the same as the impossible differential trail. It can construct 8 rounds of zero-correlation trails. Therefore, 16 rounds are analyzed at most for uLBC-128.

Table 18: An impossible differential trail of uLBC.

Position	State
Δ_f	*00000000000000000000000000000000
Round 1	0***000000000000000000000000000000
Round 2	0000*0**00000000000000*0****00000
Round 3	*0**0*****0??**0***0000**0***??
Round 4	??????*?????*????????????????*??
Round 4	****?00***0*0*?00**?0?*****0*0
Round 5	000*0**00*00*0000000*00000*0*00*
Round 6	0000000000000000*00*000000*000000
Round 7	*00000000000000000000000000000000
Δ_r	0aaa000000000000000000000000000000

4.4 Meet-in-the-Middle Attack

Partial-matching [AS08] cannot work if the number of rounds reaches full diffusion rounds in each of the forward and backward directions. The 4-round full diffusion property with our low-latency S-boxes and diffusion layer enables us to claim that any inserted key bit of K non-linearly affects all bits of the state after 4-round propagation in the forward or the backward directions for uLBC.

Thus, partial-matching can work for at most $(4 - 1) + (4 - 1) + 1 = 7$ rounds. The condition for the initial structure (IS) [SA09], also called independent biclique [BKR11], is that key differential trails in the forward direction and those in the backward direction do not share active non-linear components. For uLBC-128, since any key differential affects all 32 S-boxes after at least 4 rounds in the forward and backward directions, there is no

such differential that shares active S-boxes in more than 4 rounds. Thus, the number of rounds used for IS is bounded by 4. Assuming that the splice-and-cut technique allows an attacker to add 4 extra rounds in the worst case, at most 15-round ($4 + 4 + 7$) MitM attack may be feasible. Because a whitening key is added at the end of the encryption, we consider it is difficult to launch a 15-round attack on uLBC-128.

4.5 Integral Analysis

Integral attack [DKR97] prepares a set of plaintexts so that particular cells can contain all the values in the set and the other cells are fixed to a constant value. We search for the optimal integral distinguisher based on the division property [XZBL16, HWW20]. Modelling the division property by Mixed-Integer-Linear-Programming (MILP) we get a 10-round integral distinguisher for uLBC-128 with 127 active bits at input:

$$\begin{aligned} &C^1|A^{31}|A^{32}|A^{32}|A^{32}, \\ &\quad \downarrow 10r \\ &B^4|U^{28}|U^{32}|U^{32}|U^{32}, \end{aligned}$$

where A denotes all values in the cell appear exactly the same number, B denotes the sum of all values in the multiset is 0, C denotes the value of the bit is fixed through the multiset and U denotes no particular property exists. With this 10-round integral distinguisher, we can launch an attack with 4 rounds of key-filter at the end and one round adding to beginning with statistical method. Therefore, we can attack at most 15 rounds of uLBC-128.

We focus on differential-like and linear-like cryptanalysis and give differential cryptanalysis, boomerang attack, linear cryptanalysis, impossible differential cryptanalysis, integral cryptanalysis, and related-key cryptanalysis of uLBC algorithm and results are in Table 19. The round function of the uLBC-256 is similar to uLBC-128, but the difference is the key schedule. Therefore, the uLBC-256 focuses on the impact of key length change in the key recovery process and related key attacks. For all versions of uLBC, sufficient security redundancy is reserved.

Table 19: Summary of analysis results of uLBC

Cryptanalysis methods	Round of distinguisher	Estimated analysis round (Upper bound)		
		uLBC-128	uLBC-256	uLBC-384t
Differential Attack	8	16	18	20
Boomerang Attack	8	16	18	20
Linear Attack	8	16	18	20
Impossible differential Analysis	8	16	18	20
Meet-in-the-Middle Attack	7	15	17	19
Integral Analysis	10	14	16	17

4.6 Related-key (Related-tweak) cryptanalysis of uLBC-384t

Under the related-key scenario, we introduce an MILP model to search for the minimize number of active S-boxes, the lower bound of active S-boxes for each round is shown in Table 20. For related-tweak scenarios, the attacker has the weaker ability, the number of rounds of attack will not exceed that under the related-key scenario. With the results, we are able to prove strong bounds against related-tweak linear and differential attacks. In particular, no related tweak linear or differential distinguisher based on a characteristic is possible for 14-round uLBC-384t, which already exceeds 64 active S-boxes.

And, for the related-tweak boomerang distinguisher or differential-linear distinguisher, we can construct at most a $6 + 1 + 6 = 13$ -round distinguisher, which has 6-round upper

part and 6-round lower part, and 1-round middle part without intersection of S-boxes. The probability is at least 2^{-104} , the 14-round distinguisher also has probability lower than 2^{-128} .

According to the length of the related-key distinguisher, at most $6 + 13 + 6 = 25$ round attack can be launched when the length of key is 384-bit.

Table 20: Estimation of the number of active S-boxes for uLBC-384t under related-tweak

Round	1	2	3	4	5	6	7	8	9	10	11	12	13	14
# of active S-boxes	0	0	1	4	8	13	23	32	35	39	44	51	60	>64

5 Implementation

Since our target construction is a low-latency block cipher, thus we want to minimize the latency of our block cipher, uLBC-128 is fully unrolled (that optimizes the signal delay from the input to output ports) and has its latency measured as a combinatorial circuit. For comparison, the latency of other ciphers with no less than 128 bits of security and block size are also included, namely QARMA₉-128, QARMA₁₁-128, Midori-128, SPEEDY-6, and AES-128. As we can see in Table 21, uLBC-128 has the best area performance among all the ciphers and the minimum delay among the 128-bit block ciphers. In this section, we discuss the result of uLBC-128, and the implementation result of uLBC family is in Appendix C.

The delay and area of the ciphers are measured with *Synopsys Design Compiler S-2021.06-SP3* and 2 open process libraries (Nangate 15nm and Nangate 45nm) and 2 commercial process libraries (TSMC 28nm and UMC 55nm). We follow the evaluation framework outlined in [BIL⁺21, ABD⁺23], using the `set_max_delay/set_max_area` instructions to reduce the delay/area. Thus, we give two results, one is the area optimized implementation and the other is the latency optimized implementation.

It can be seen that for both libraries, uLBC-128 has the second shortest latency (the fastest is SPEEDY-6), and the smallest area. However, since SPEEDY-7-192 is fully broken by Boura *et al.* [BDHBNP23], we believe its advantage in performance should be treated with caution. Compared with Midori-128, uLBC-128 has a delay of 11%~15% shorter and uLBC-128 has excellent AT performance, the best performance except SPEEDY-6, and even the best performance in UMC55nm.

6 Conclusion

In this paper, we propose a new low-latency 128-bit block cipher, called uLBC-128, which offers a reduced implementation cost. Our experimental results consistently demonstrate that uLBC-128 exhibits lower latency and requires less area compared to QARMA₉-128, Midori128, and AES-128. Furthermore, uLBC-128 presents a higher throughput-to-area ratio when compared to these three ciphers. While the SPEEDY-6 cipher offers the lowest latency among the tested ciphers, its security arguments fall short, particularly when considering that Boura *et al.* have successfully broken the full round of SPEEDY-7-192. This raises concerns about the security of the 7-round SPEEDY. Thus, it is crucial to provide evidence of the security of our newly designed cipher. The uLBC block cipher offers robust security arguments against differential and linear attacks.

In terms of future directions, it would be interesting to extend our design by applying the new methods for low-latency 4-bit S-boxes to larger S-boxes, such as those with 5-8 bits. Additionally, exploring the efficient implementation of the cipher using instruction set architecture such as SIMD and AVX2 for software performance and side-channel security would also be valuable areas to investigate.

Table 21: Area and Latency Optimized Cipher Performance. AT denotes area-time product, which we set to $AT = \text{area}(\text{GE}) \cdot \text{Delay}(\text{ns})/1000$.

Platform	Cipher	Area Optimized				Latency Optimized			
		Area μm^2	GE	Delay ns	AT	Area μm^2	GE	Delay ns	AT
NanGate 15nm	AES	24333	123764	1.292	16.0	32637	166000	0.785	13.0
	Midori128	5114	26011	0.889	2.3	6839	34783	0.627	2.2
	QARMA ₉ -128	6237	31723	0.950	3.0	9082	46193	0.650	3.0
	QARMA ₁₁ -128	7503	38162	1.170	4.5	10710	54475	0.780	4.2
	SPEEDY-6	5666	28819	0.589	1.7	7576	38532	0.370	1.4
	uLBC-128	4349	22120	0.740	1.6	5742	29204	0.551	1.6
TSMC 28nm	AES	55702	110520	3.13	34.6	103337	205033	1.57	32.2
	Midori128	12936	25667	2.43	6.2	34522	68496	1.22	8.4
	QARMA ₉ -128	15358	30472	2.37	7.2	42191	83713	1.29	10.8
	QARMA ₁₁ -128	18304	36318	2.83	10.3	48495	96220	1.57	15.1
	SPEEDY-6	13938	27655	1.33	3.7	26616	52809	0.71	3.7
	uLBC-128	10933	21693	1.98	4.3	24897	49399	1.09	5.4
NanGate 45nm	AES	91710	114925	14.38	165.3	120052	150441	8.12	122.2
	Midori128	18907	23693	9.06	21.5	24646	30885	6.73	20.8
	QARMA ₉ -128	22929	28733	10.37	29.8	33803	42360	6.87	29.1
	QARMA ₁₁ -128	27510	34474	12.54	43.2	40297	50497	8.27	41.8
	SPEEDY-6	21079	26415	5.89	15.6	27441	34387	3.98	13.7
	uLBC-128	15955	19994	7.63	15.3	19922	24964	5.74	14.3
UMC 55nm	AES	123037	109854	14.21	156.1	294367	262828	3.74	98.3
	Midori128	26645	23790	7.24	17.2	111906	99916	2.83	28.3
	QARMA ₉ -128	29173	26047	7.45	28.2	114481	102215	2.59	38.0
	QARMA ₁₁ -128	34831	31099	9.06	28.2	137188	122489	3.10	26.5
	SPEEDY-6	28859	25767	6.68	13.2	108594	96959	1.56	15.1
	uLBC-128	22545	20130	5.60	11.3	78362	69966	2.23	15.6

References

- [ABD⁺23] Roberto Avanzi, Subhadeep Banik, Orr Dunkelman, Maria Eichlseder, Shibam Ghosh, Marcel Nageler, and Francesco Regazzoni. The qarmav2 family of tweakable block ciphers. *IACR Trans. Symmetric Cryptol.*, 2023(3):25–73, 2023. URL: <https://doi.org/10.46586/tosc.v2023.i3.25-73>, doi:10.46586/TOSC.V2023.I3.25-73.
- [AS08] Kazumaro Aoki and Yu Sasaki. Preimage attacks on one-block md4, 63-step MD5 and more. In Roberto Maria Avanzi, Liam Keliher, and Francesco Sica, editors, *Selected Areas in Cryptography, 15th International Workshop, SAC 2008, Sackville, New Brunswick, Canada, August 14-15, Revised Selected Papers*, volume 5381 of *Lecture Notes in Computer Science*, pages 103–119. Springer, 2008. doi:10.1007/978-3-642-04159-4_7.
- [Ava17] Roberto Avanzi. The QARMA block cipher family. almost MDS matrices over rings with zero divisors, nearly symmetric even-mansour constructions with non-involutory central rounds, and search heuristics for low-latency s-boxes. *IACR Trans. Symmetric Cryptol.*, 2017(1):4–44, 2017. doi:10.13154/tosc.v2017.i1.4-44.
- [BBI⁺15] Subhadeep Banik, Andrey Bogdanov, Takanori Isobe, Kyoji Shibutani, Harunaga Hiwatari, Toru Akishita, and Francesco Regazzoni. Midori: A Block Cipher for Low Energy. In Tetsu Iwata and Jung Hee Cheon, editors, *Advances in Cryptology - ASIACRYPT 2015 - 21st International Conference on the Theory and Application of Cryptology and Information*

- 674 *Security, Auckland, New Zealand, November 29 - December 3, 2015, Pro-*
675 *ceedings, Part II*, volume 9453 of *Lecture Notes in Computer Science*, pages
676 411–436. Springer, 2015. doi:10.1007/978-3-662-48800-3_17.
- 677 [BBS99] Eli Biham, Alex Biryukov, and Adi Shamir. Cryptanalysis of skipjack
678 reduced to 31 rounds using impossible differentials. In Jacques Stern, editor,
679 *Advances in Cryptology - EUROCRYPT '99, International Conference on*
680 *the Theory and Application of Cryptographic Techniques, Prague, Czech*
681 *Republic, May 2-6, 1999, Proceeding*, volume 1592 of *Lecture Notes in*
682 *Computer Science*, pages 12–23. Springer, 1999. doi:10.1007/3-540-489
683 10-X_2.
- 684 [BCG⁺12] Julia Borghoff, Anne Canteaut, Tim Güneysu, Elif Bilge Kavun, Miroslav
685 Knezevic, Lars R. Knudsen, Gregor Leander, Ventzislav Nikov, Christof
686 Paar, Christian Rechberger, Peter Rombouts, Søren S. Thomsen, and Tolga
687 Yalçın. Prince – a low-latency block cipher for pervasive computing applica-
688 tions. In Xiaoyun Wang and Kazue Sako, editors, *Advances in Cryptology*
689 *– ASIACRYPT 2012*, pages 208–225, Berlin, Heidelberg, 2012. Springer
690 Berlin Heidelberg.
- 691 [BCLR17] Christof Beierle, Anne Canteaut, Gregor Leander, and Yann Rotella. Prov-
692 ing resistance against invariant attacks: How to choose the round constants.
693 In Jonathan Katz and Hovav Shacham, editors, *Advances in Cryptology –*
694 *CRYPTO 2017*, pages 647–678, Cham, 2017. Springer International Pub-
695 lishing.
- 696 [BDD⁺23] Yanis Belkheyar, Joan Daemen, Christoph Dobraunig, Santosh Ghosh,
697 and Shahram Rasoolzadeh. Bipbip: A low-latency tweakable block ci-
698 pher with small dimensions. *IACR Trans. Cryptogr. Hardw. Embed. Syst.*,
699 2023(1):326–368, 2023. doi:10.46586/tches.v2023.i1.326-368.
- 700 [BDHBNP23] Christina Boura, Nicolas David, Rachelle Heim Boissier, and María Naya-
701 Plasencia. Better steady than speedy: Full break of speedy-7-192. In
702 Carmit Hazay and Martijn Stam, editors, *Advances in Cryptology – EU-*
703 *ROCRYPT 2023*, pages 36–66, Cham, 2023. Springer Nature Switzerland.
- 704 [BEK⁺21] Dušan Božilov, Maria Eichlseder, Miroslav Knezevic, Baptiste Lambin,
705 Gregor Leander, Thorben Moos, Ventzislav Nikov, Shahram Rasoolzadeh,
706 Yosuke Todo, and Friedrich Wiemer. Princev2 - more security for (al-
707 most) no overhead. In Orr Dunkelman, Michael J. Jacobson, Jr., and
708 Colin O’Flynn, editors, *Selected Areas in Cryptography*, Lecture Notes
709 in Computer Science, pages 483–511. Springer, 2021. 27th International
710 Conference on Selected Areas in Cryptography : SAC 2020, SAC 2020 ;
711 Conference date: 19-10-2020 Through 23-10-2020. doi:10.1007/978-3-0
712 30-81652-0_19.
- 713 [BGLS19] Zhenzhen Bao, Jian Guo, San Ling, and Yu Sasaki. Peigen – a platform for
714 evaluation, implementation, and generation of s-boxes. *IACR Transactions*
715 *on Symmetric Cryptology*, 2019(1):330–394, Mar. 2019. URL: <https://to>
716 [sc.iacr.org/index.php/ToSC/article/view/7406](https://tosc.iacr.org/index.php/ToSC/article/view/7406), doi:10.13154/tos
717 [c.v2019.i1.330-394](https://tosc.iacr.org/index.php/ToSC/article/view/7406).
- 718 [BIL⁺21] Subhadeep Banik, Takanori Isobe, Fukang Liu, Kazuhiko Minematsu, and
719 Kosei Sakamoto. Orthros: A Low-Latency PRF. *IACR Transactions on*
720 *Symmetric Cryptology*, 2021(1):37–77, Mar. 2021. URL: <https://tosc.i>

- acr.org/index.php/ToSC/article/view/8833, doi:10.46586/tosc.v2021.i1.37-77.
- [BJK⁺16] Christof Beierle, Jérémy Jean, Stefan Kölbl, Gregor Leander, Amir Moradi, Thomas Peyrin, Yu Sasaki, Pascal Sasdrich, and Siang Meng Sim. The SKINNY family of block ciphers and its low-latency variant MANTIS. In Matthew Robshaw and Jonathan Katz, editors, *Advances in Cryptology - CRYPTO 2016 - 36th Annual International Cryptology Conference, Santa Barbara, CA, USA, August 14-18, 2016, Proceedings, Part II*, volume 9815 of *Lecture Notes in Computer Science*, pages 123–153. Springer, 2016. doi:10.1007/978-3-662-53008-5_5.
- [BKR11] Andrey Bogdanov, Dmitry Khovratovich, and Christian Rechberger. Biclique cryptanalysis of the full AES. In Dong Hoon Lee and Xiaoyun Wang, editors, *Advances in Cryptology - ASIACRYPT 2011 - 17th International Conference on the Theory and Application of Cryptology and Information Security, Seoul, South Korea, December 4-8, 2011. Proceedings*, volume 7073 of *Lecture Notes in Computer Science*, pages 344–371. Springer, 2011. doi:10.1007/978-3-642-25385-0_19.
- [DKR97] Joan Daemen, Lars R. Knudsen, and Vincent Rijmen. The block cipher square. In Eli Biham, editor, *Fast Software Encryption, 4th International Workshop, FSE '97, Haifa, Israel, January 20-22, 1997, Proceedings*, volume 1267 of *Lecture Notes in Computer Science*, pages 149–165. Springer, 1997. doi:10.1007/BFb0052343.
- [DKS10] Orr Dunkelman, Nathan Keller, and Adi Shamir. A practical-time related-key attack on the KASUMI cryptosystem used in GSM and 3g telephony. In Tal Rabin, editor, *Advances in Cryptology - CRYPTO 2010, 30th Annual Cryptology Conference, Santa Barbara, CA, USA, August 15-19, 2010. Proceedings*, volume 6223 of *Lecture Notes in Computer Science*, pages 393–410. Springer, 2010. doi:10.1007/978-3-642-14623-7_21.
- [GJN⁺15] Jian Guo, Jérémy Jean, Ivica Nikolić, Kexin Qiao, Yu Sasaki, and Siang Meng Sim. Invariant Subspace Attack Against Full Midori64. Cryptology ePrint Archive, Paper 2015/1189, 2015. <https://eprint.iacr.org/2015/1189>. URL: <https://eprint.iacr.org/2015/1189>.
- [GL16] David Gérardt and Pascal Lafourcade. Related-Key Cryptanalysis of Midori. In Orr Dunkelman and Somitra Kumar Sanadhya, editors, *Progress in Cryptology – INDOCRYPT 2016*, pages 287–304, Cham, 2016. Springer International Publishing.
- [HWW20] Kai Hu, Qingju Wang, and Meiqin Wang. Finding bit-based division property for ciphers with complex linear layers. *IACR Trans. Symmetric Cryptol.*, 2020(1):396–424, 2020. doi:10.13154/tosc.v2020.i1.396-424.
- [JNP14] Jérémy Jean, Ivica Nikolić, and Thomas Peyrin. Tweaks and keys for block ciphers: The tweakkey framework. In Palash Sarkar and Tetsu Iwata, editors, *Advances in Cryptology – ASIACRYPT 2014*, pages 274–288, Berlin, Heidelberg, 2014. Springer Berlin Heidelberg.
- [LH94] Susan K. Langford and Martin E. Hellman. Differential-linear cryptanalysis. In Yvo G. Desmedt, editor, *Advances in Cryptology — CRYPTO '94*, pages 17–25, Berlin, Heidelberg, 1994. Springer Berlin Heidelberg.

- [LMMR21] Gregor Leander, Thorben Moos, Amir Moradi, and Shahram Rasoolzadeh. The SPEEDY family of block ciphers engineering an ultra low-latency cipher from gate level for secure processor architectures. *IACR Trans. Cryptogr. Hardw. Embed. Syst.*, 2021(4):510–545, 2021. doi:10.46586/tches.v2021.i4.510-545.
- [LS16] Meicheng Liu and Siang Meng Sim. Lightweight mds generalized circulant matrices. In Thomas Peyrin, editor, *Fast Software Encryption*, pages 101–120, Berlin, Heidelberg, 2016. Springer Berlin Heidelberg.
- [Ras22] Shahram Rasoolzadeh. Low-latency boolean functions and bijective s-boxes. *IACR Trans. Symmetric Cryptol.*, 2022(3):403–447, 2022. URL: <https://doi.org/10.46586/tosc.v2022.i3.403-447>, doi:10.46586/TOSC.V2022.I3.403-447.
- [SA09] Yu Sasaki and Kazumaro Aoki. Finding preimages in full MD5 faster than exhaustive search. In Antoine Joux, editor, *Advances in Cryptology - EUROCRYPT 2009, 28th Annual International Conference on the Theory and Applications of Cryptographic Techniques, Cologne, Germany, April 26-30, 2009. Proceedings*, volume 5479 of *Lecture Notes in Computer Science*, pages 134–152. Springer, 2009. doi:10.1007/978-3-642-01001-9_8.
- [SSH99] I. Sutherland, Bob Sproull, and David Harris. Logical Effort: Designing Fast CMOS Circuits. 1999.
- [TAY17] Mohamed F. Tolba, Ahmed Abdelkhalek, and Amr M. Youssef. Improved Multiple Impossible Differential Cryptanalysis of Midori128. *IEICE Trans. Fundam. Electron. Commun. Comput. Sci.*, 100-A:1733–1737, 2017.
- [XZBL16] Zejun Xiang, Wentao Zhang, Zhenzhen Bao, and Dongdai Lin. Applying MILP method to searching integral distinguishers based on division property for 6 lightweight block ciphers. In Jung Hee Cheon and Tsuyoshi Takagi, editors, *Advances in Cryptology - ASIACRYPT 2016 - 22nd International Conference on the Theory and Application of Cryptology and Information Security, Hanoi, Vietnam, December 4-8, 2016, Proceedings, Part I*, volume 10031 of *Lecture Notes in Computer Science*, pages 648–678, 2016. doi:10.1007/978-3-662-53887-6_24.

A Test Vectors

A.1 uLBC-128

- | | |
|------------|---------------------------------------|
| Plaintext | : 00000000 00000000 00000000 00000000 |
| 1. Key | : 00000000 00000000 00000000 00000000 |
| Ciphertext | : e6b6b249 c25994a4 25c480b8 52f4899a |
| Plaintext | : 2d33f9e1 4c935dc3 150e07ef 14909c24 |
| 2. Key | : 2f5a39f6 5ef0d47b 4c2ae6b5 21fb0a08 |
| Ciphertext | : 5384bf47 150a1f05 059c1e70 4894ca21 |

801 A.2 uLBC-256

1.	Plaintext	: 00000000 00000000 00000000 00000000
	Key	: 00000000 00000000 00000000 00000000
		: 00000000 00000000 00000000 00000000
	Ciphertext	: fb0be725 0cfd59a9 d2712fb1 899ff396
2.	Plaintext	: 2d643085 0cad830b 53764dad 7be5c3a9
	Key	: 2990cbcb 613391be 00a582cd 722e610f
		: 42e65a59 2135ab5f 4a85fb01 222224e0
	Ciphertext	: b6cfa6a5 90dba5ea a5a7004e 5a8339f4

803 A.3 uLBC-384t

1.	Plaintext	: 00000000 00000000 00000000 00000000
	Key	: 00000000 00000000 00000000 00000000
		: 00000000 00000000 00000000 00000000
	Tweak	: 00000000 00000000 00000000 00000000
	Ciphertext	: 3255cc94 fd3b23ef 6beb0bd9 88fbb738
2.	Plaintext	: 2d7d941a 0f864e4b 3a7e25e3 2fe97815
	Key	: 08cac346 37da2d2c 54cbc059 0cc13e8d
		: 62f1a185 65a35582 4642fdef 582d79f2
	Tweak	: 11a12c18 6c55e4b2 7b1999ae 4db8ada5
	Ciphertext	: e7646a3f af01a7c3 3f290d33 92a8a3dc

805 B Differential trails with minimize number of active 806 sboxes for uLBC-128

807 Table 22 shows the differential trails with the minimum number of active S-boxes for
808 uLBC-128 with 3 to 7 Rounds.

809 C Implementation of uLBC Family

810 Table 23 shows the implementation of uLBC family.

Table 22: Differential trails with the minimized number of active S-boxes.

# of Round	Position	State
$3r$	Δ_{in}	000000000c0020000000000000900000
	Round 1	00000000000000060000000000000000
	Round 2	22200000000000000000000000000000
	Δ_{out}	06661011000000000000550500000000
$4r$	Δ_{in}	0002000000000000200000000002000
	Round 1	00000000000000000000000000900000
	Round 2	00000000000660600000000000000000
	Round 3	ccc0000000009990000202200000000
	Δ_{out}	3ffc6066110111100555563500006606
$5r$	Δ_{in}	00040010120020000100004200000400
	Round 1	00000000000400100000000040000000
	Round 2	00000000000000000000000000000800
	Round 3	00000000202200000000000000000000
	Round 4	00000000088800001101000000001110
	Δ_{out}	0000aa82404480882202088881992220
$6r$	Δ_{in}	0000000000000000800000100108000
	Round 1	0000000000000110000000022000000
	Round 2	08080000000000000000000000001010
	Round 3	00000880000000000000000002002000
	Round 4	0000000000000002022111022020888
	Round 5	01100088088000002002800800001100
	Δ_{out}	00009198323102028080891926460000
$7r$	Δ_{in}	0000000000000000100000200102000
	Round 1	0000000000000110000000011000000
	Round 2	01010000000000000000000000001010
	Round 3	00000220000000000000000002002000
	Round 4	0000000000000001011111011010111
	Round 5	01100088011000001001400400008800
	Round 6	00000101020204044040808020200000
	Δ_{out}	00880000008801108800440000001001
$8r$	Δ_{in}	10000001800000000020008002000000
	Round 1	00010000020000000000000000000000
	Round 2	0000000000080880000000011100000
	Round 3	08080000000020200000000000002313
	Round 4	00000220101101111110000080081101
	Round 5	00110000110000001000008000010200
	Round 6	00000000000810110000010044400000
	Round 7	0808000000080800000000000004c00
	Δ_{out}	0000808860660222000000008cc42202

Table 23: Area and Latency Optimized Cipher Performance (with uLBC family). AT denotes area-time product, which we set to $AT = \text{area}(\text{GE}) \cdot \text{Delay}(\text{ns})/1000$.

Platform	Cipher	Area Optimized				Latency Optimized			
		Area μm^2	GE	Delay ns	AT	Area μm^2	GE	Delay ns	AT
NanGate 15nm	AES	24333	123764	1.292	16.0	32637	166000	0.785	13.0
	Midori128	5114	26011	0.889	2.3	6839	34783	0.627	2.2
	QARMA ₉ -128	6237	31723	0.950	3.0	9082	46193	0.650	3.0
	QARMA ₁₁ -128	7503	38162	1.170	4.5	10710	54475	0.780	4.2
	SPEEDY-6	5666	28819	0.589	1.7	7576	38532	0.370	1.4
	uLBC-128	4349	22120	0.740	1.6	5742	29204	0.551	1.6
	uLBC-256	6146	31262	0.964	3.0	8052	40953	0.681	2.8
	uLBC-384t	8863	45078	1.302	5.9	11527	58628	0.927	5.4
TSMC 28nm	AES	55702	110520	3.13	34.6	103337	205033	1.57	32.2
	Midori128	12936	25667	2.43	6.2	34522	68496	1.22	8.4
	QARMA ₉ -128	15358	30472	2.37	7.2	42191	83713	1.29	10.8
	QARMA ₁₁ -128	18304	36318	2.83	10.3	48495	96220	1.57	15.1
	SPEEDY-6	13938	27655	1.33	3.7	26616	52809	0.71	3.7
	uLBC-128	10933	21693	1.98	4.3	24897	49399	1.09	5.4
	uLBC-256	15913	31574	2.32	7.3	36150	71726	1.34	9.6
	uLBC-384t	22961	45558	3.16	14.4	47189	93628	1.84	17.2
NanGate 45nm	AES	91710	114925	14.38	165.3	120052	150441	8.12	122.2
	Midori128	18907	23693	9.06	21.5	24646	30885	6.73	20.8
	QARMA ₉ -128	22929	28733	10.37	29.8	33803	42360	6.87	29.1
	QARMA ₁₁ -128	27510	34474	12.54	43.2	40297	50497	8.27	41.8
	SPEEDY-6	21079	26415	5.89	15.6	27441	34387	3.98	13.7
	uLBC-128	15955	19994	7.63	15.3	19922	24964	5.74	14.3
	uLBC-256	22704	28451	9.83	28.0	28165	35295	7.15	25.2
	uLBC-384t	32833	41144	13.3	54.7	39983	50104	9.62	48.2
UMC 55nm	AES	123037	109854	14.21	156.1	294367	262828	3.74	98.3
	Midori128	26645	23790	7.24	17.2	111906	99916	2.83	28.3
	QARMA ₉ -128	29173	26047	7.45	28.2	114481	102215	2.59	38.0
	QARMA ₁₁ -128	34831	31099	9.06	28.2	137188	122489	3.10	26.5
	SPEEDY-6	28859	25767	6.68	13.2	108594	96959	1.56	15.1
	uLBC-128	22545	20130	5.60	11.3	78362	69966	2.23	15.6
	uLBC-256	31609	28223	7.30	20.6	105104	93843	2.77	26.0
	uLBC-384t	45611	40724	10.17	41.4	146099	130446	3.79	49.4